



Temporizadore S

clase 7 - Fecha: 19/05/2026

Objetivos de la clase

- Explicar qué problema resuelve un temporizador.
- Relacionar frecuencia, período y tiempo de conteo.
- Identificar los registros principales de un temporizador STM32.
- Configurar un temporizador como base de tiempo.
- Generar una señal PWM y su ciclo de trabajo.
- Configuración de registros.

Temporizadores, interrupciones y PWM

Temporizadores, interrupciones externas y PWM



TEMPORIZADORES

Generación de retardos,
medición de tiempo y
eventos periódicos.



INTERRUPCIONES EXTERNAS

Respuesta inmediata a
eventos externos mediante
líneas EXTI.



PWM

Generación de señales
PWM para control de
potencia y actuadores.



PLACA:
NUCLEO-F446RE



MCU:
STM32F446RE



STM32

Desarrolla. Conecta. Innova.

¿Delay o timer?

HAL_Delay()	Timer por hardware
Bloquea el programa	Cuenta en paralelo a la CPU
Sirve para ejemplos simples	Sirve para eventos precisos y periódicos
Tiene granularidad de milisegundos	Puede trabajar con resolución mucho menor
La CPU queda esperando	La CPU puede ejecutar otras tareas
No genera señales por sí solo	Puede generar interrupciones, PWM o capturas

Frecuencia y período

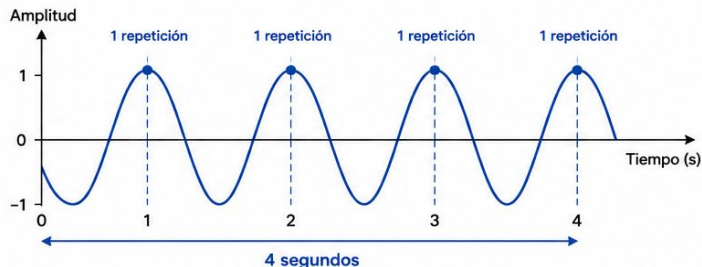
FRECUENCIA (f)

La frecuencia indica **cuántas veces se repite la señal en un segundo**.

$$f = \frac{1}{T}$$

Unidad: $\text{Hz} = \frac{1}{\text{s}}$

La frecuencia indica cuántas veces se repite la señal en un segundo.



Ejemplos

¿Cuántas veces se repite la señal?	Tiempo	Frecuencia (f)
1 vez	1 segundo	1 Hz
2 veces	1 segundo	2 Hz
4 veces	1 segundo	4 Hz
10 veces	1 segundo	10 Hz
1000 veces	1 segundo	1 kHz
10000 veces	1 segundo	10 kHz



En este ejemplo: la señal se repite 4 veces en 4 segundos $\rightarrow f = \frac{4 \text{ veces}}{4 \text{ s}} = 1 \text{ Hz}$ (1 repetición por segundo)

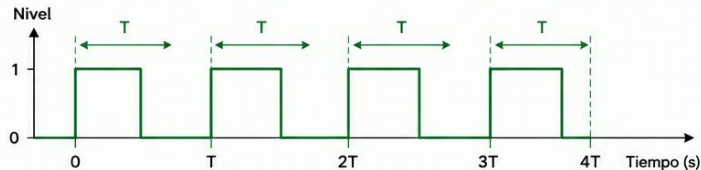
PERÍODO (T)

El período indica cuánto tiempo tarda la señal en completar 1 repetición.

$$T = \frac{1}{f}$$

Unidad: s (segundos)

El período es el tiempo que tarda la señal en completar 1 repetición.



Si $f = 2 \text{ Hz}$, entonces $T = \frac{1}{f} = \frac{1}{2} \text{ s} = 0.5 \text{ s} = 500 \text{ ms}$

Relación entre frecuencia y período

Son magnitudes inversamente proporcionales.

$$f = \frac{1}{T} \quad \text{y} \quad T = \frac{1}{f}$$

Si la frecuencia aumenta, el período disminuye.

Si la frecuencia disminuye, el período aumenta.

$\uparrow f \rightarrow \downarrow T$ $\downarrow f \rightarrow \uparrow T$



RESUMEN

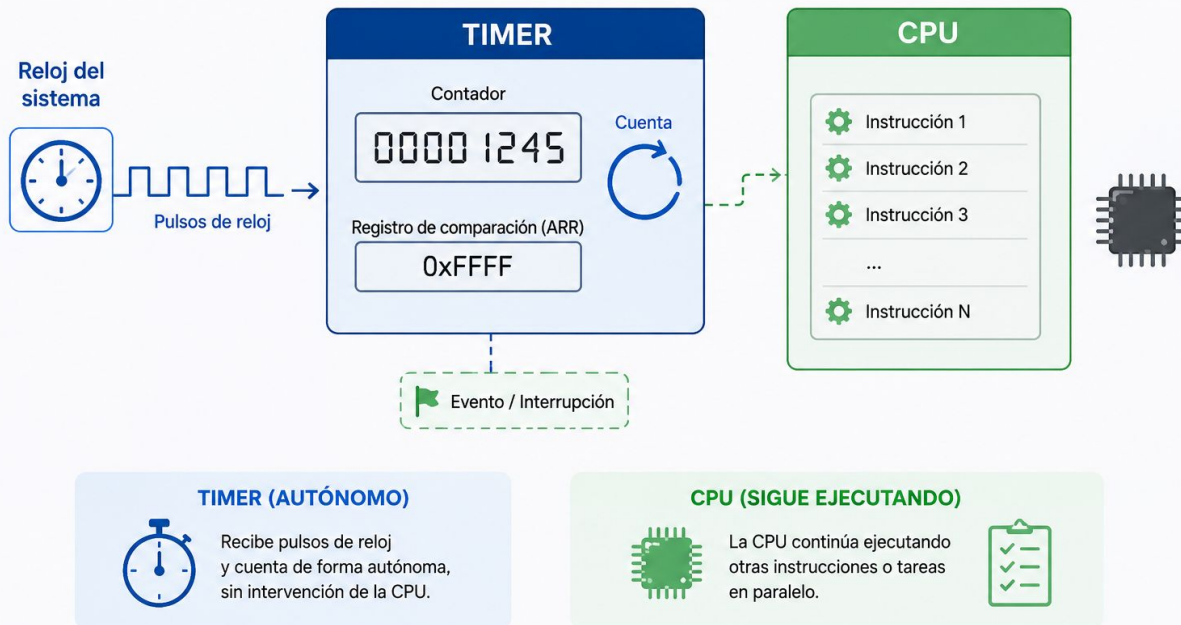
- Frecuencia (f):** cuántas veces se repite la señal en un segundo. Unidad: Hz (1/s)
- Período (T):** tiempo que tarda la señal en completar 1 repetición. Unidad: segundos (s)



A mayor frecuencia, menor período.

¿Qué es un timer?

El **timer** cuenta pulsos de reloj de manera **autónoma**.
Mientras el timer cuenta, la **CPU** puede seguir ejecutando otras instrucciones.

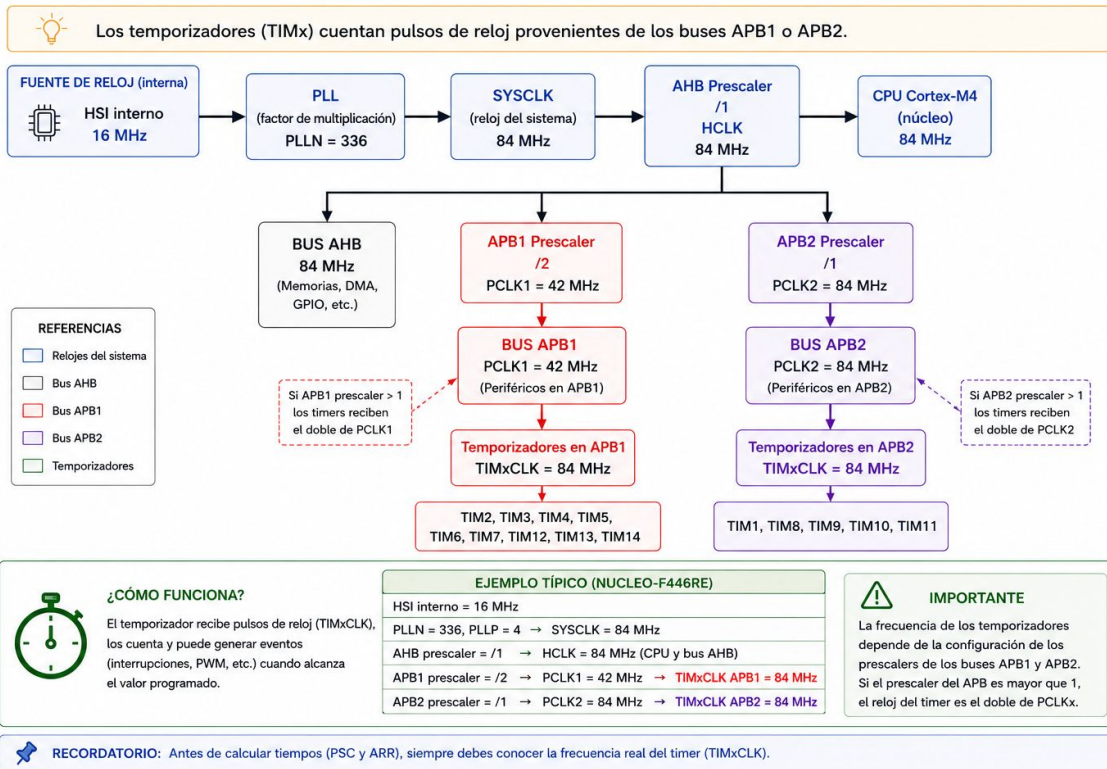


Timers disponibles en STM32F446RE

Tipo	Timers	Uso típico
Avanzados	TIM1, TIM8	PWM avanzado, control de motores, salidas complementarias
Generales completos	TIM2, TIM3, TIM4, TIM5	Base de tiempo, interrupciones, PWM, captura/comparación
Generales compactos	TIM9 a TIM14	PWM, comparación, bases de tiempo simples
Básicos	TIM6, TIM7	Base de tiempo interna, disparo de DAC
SysTick	Timer del núcleo Cortex-M	Base temporal de HAL
Watchdogs	IWDG, WWDG	Supervisión del sistema

¿Qué reloj utiliza cada timer?

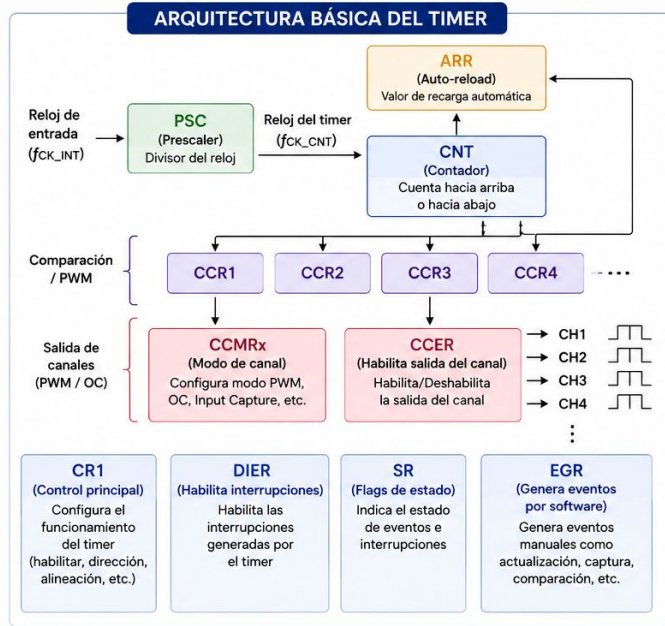
STM32F446RE – Relojes, Buses y Temporizadores



Registros principales de un timer



Los timers de STM32 están configurados mediante registros que controlan el conteo, las comparaciones, las interrupciones y las salidas.



Registro	Función
PSC	Prescaler Divide la frecuencia del reloj de entrada del timer.
ARR	Auto-reload Define el valor al que se recarga el contador cuando ocurre un evento de actualización.
CNT	Contador Registra el valor actual del contador (incrementa o decrementa).
CR1	Control principal Controla el funcionamiento general del timer.
DIER	Habilita interrupciones Habilita las interrupciones del timer.
SR	Flags de estado Muestra el estado de eventos e interrupciones.
EGR	Genera eventos por software Permite generar eventos de actualización, captura, comparación, etc., por software.
CCR1..CCR4	Comparación / PWM Registros de comparación para generar PWM o capturar eventos.
CCMRx	Modo de canal Configura el modo de operación del canal: PWM, Output Compare, Input Capture, etc.
CCER	Habilita salida del canal Habilita o deshabilita la salida física del canal.



Notas: Estos registros pueden variar ligeramente según el modelo de timer (TIMx) y la familia STM32.



Reloj y conteo



Comparación



Canales y salidas



Control y estado

Tamaño de los temporizadores APB1

Timer	Tamaño del contador	Valor máximo de ARR
TIM2, TIM5	32 bits	4.294.967.295
TIM3, TIM4	16 bits	65.535
TIM6, TIM7	16 bits	65.535

Ejemplo 1

CASO 1 — TIMER DE 32 BITS SIN PRESCALER

Objetivo: generar un evento cada 1 segundo ($T_{evento} = 1\text{ s}$)

1 DATOS DEL PROBLEMA

- Frecuencia del timer: $f_{TIM} = 84\text{ MHz}$
- Prescaler: $PSC = 0$ (sin prescaler)
- Queremos: $f_{evento} = 1\text{ Hz}$ ($T_{evento} = 1\text{ s}$)



IDEA CLAVE

Si $PSC = 0$, el timer no divide la frecuencia.
Por lo tanto, el contador (CNT) avanza a la misma frecuencia del reloj del timer.

2 FÓRMULA DEL TIMER

$$f_{evento} = \frac{f_{TIM}}{(PSC + 1) \cdot (ARR + 1)}$$

i Los registros usan +1 porque el conteo incluye el valor cero.

3 FRECUENCIA DEL CONTADOR (CNT)

Como $PSC = 0$:

$$f_{CNT} = \frac{f_{TIM}}{PSC + 1} = \frac{84\text{ MHz}}{0 + 1} = 84\text{ MHz}$$

El contador avanza a 84 MHz → 1 cuenta cada 11.90 ns

4 CÁLCULO DE ARR PARA OBTENER 1 SEGUNDO

Queremos que ocurra un evento cada 1 segundo ($T_{evento} = 1\text{ s}$):

$$f_{evento} = \frac{f_{TIM}}{(PSC + 1) \cdot (ARR + 1)} \rightarrow 1 = \frac{84\text{ MHz}}{(0 + 1) \cdot (ARR + 1)} \rightarrow ARR + 1 = \frac{84\text{ MHz}}{1} = 84.000.000$$
$$\downarrow$$
$$ARR = 84.000.000 - 1 = 83.999.999$$

5 VERIFICACIÓN

Verificamos que con estos valores el evento ocurre cada 1 s:

$$f_{evento} = \frac{f_{TIM}}{(PSC + 1) \cdot (ARR + 1)}$$
$$= \frac{84\text{ MHz}}{(0 + 1) \cdot (83.999.999 + 1)}$$
$$= \frac{84.000.000}{84.000.000} = 1\text{ Hz} \quad \checkmark$$

6 VALORES FINALES DE LOS REGISTROS

```
TIM2->PSC = 0;  
TIM2->ARR = 83999999; // 83.999.999
```



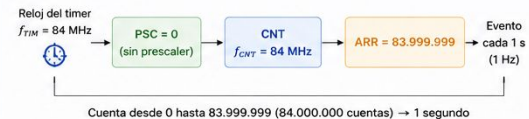
¿ENTRA EN UN TIMER DE 32 BITS?

El valor máximo que puede almacenar un timer de 32 bits es $2^{32} - 1 = 4.294.967.295$

Nuestro $ARR = 83.999.999 < 4.294.967.295 \quad \checkmark$

→ Por lo tanto, este valor SÍ entra en un timer de 32 bits.

7 RESUMEN VISUAL



CONCLUSIÓN: Con $PSC = 0$ y $ARR = 83.999.999$ en un timer de 32 bits con $f_{TIM} = 84\text{ MHz}$, se genera un evento exactamente cada 1 segundo.

Ejemplo 2

CASO 2 — TIMER DE 16 BITS CON PRESCALER



Objetivo: generar un evento cada 1 segundo ($T_{\text{evento}} = 1 \text{ s}$)

1. ¿POR QUÉ NO PODEMOS USAR CON PRESCALER?

- Timer de 16 bits → ARR máximo = 65.535
- Para 1 segundo sin prescaler necesitamos:

$$ARR + 1 = 84.000.000 \rightarrow \text{ARR} = 83.999.999$$

- Ese valor **NO** entra en un registro de 16 bits.



Por eso, para timers de 16 bits necesitamos dividir el reloj con PSC.

2. EJEMPLO PRÁCTICO CON PRESCALER

2.1 Elegimos f_{CNT}

Hacemos que el contador avance a:

$$f_{CNT} = 10 \text{ kHz}$$

Es decir:

$$1 \text{ cuenta} = 0.1 \text{ ms}$$

2.2 Cálculo de PSC

Entonces:

$$PSC + 1 = \frac{84 \text{ MHz}}{10 \text{ kHz}} = 8400$$



$$PSC = 8399$$

2.3 Cálculo de ARR

Para 1 segundo:

$$ARR + 1 = 10000$$



$$ARR = 9999$$

3. VALORES FINALES DE LOS REGISTROS



TIM3->PSC = 8399;

TIM3->ARR = 9999;

4. VERIFICACIÓN

$$\begin{aligned} f_{\text{evento}} &= \frac{f_{TIM}}{(PSC + 1) \cdot (ARR + 1)} = \frac{84 \text{ MHz}}{(8399 + 1) \cdot (9999 + 1)} \\ &= \frac{84 \text{ MHz}}{8400 \cdot 10000} = \frac{84 \text{ MHz}}{84.000.000} = 1 \text{ Hz} \quad \checkmark \end{aligned}$$



Conclusión: Usando un prescaler de 8399 y un ARR de 9999, un timer de 16 bits con $f_{TIM} = 84 \text{ MHz}$ genera un evento exactamente cada 1 segundo.

Ahora vamos a implementar el mismo temporizador de dos formas:

1. Por polling.
2. Por interrupción.

Comparación entre ambos proyectos

Aspecto	Polling	Interrupción
¿Quién detecta el evento?	El while(1) consulta SR	El NVIC llama al handler
Registro clave	SR	DIER, SR, NVIC
CPU	Debe consultar repetidamente	Puede ejecutar otra tarea
Complejidad	Menor	Mayor
Uso didáctico	Entender flags	Entender interrupciones

Descanso

¿Qué es PWM?

PWM — PULSE WIDTH MODULATION

Modulación por ancho de pulso



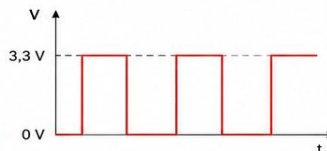
¿QUÉ ES PWM?

Una señal PWM es una señal digital que alterna entre dos niveles:

- Nivel bajo → 0 V
- Nivel alto → 3,3 V



La señal no cambia su amplitud. Lo que cambia es el **tiempo** que permanece en alto dentro de cada período.

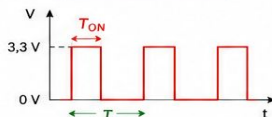


PARÁMETROS PRINCIPALES

Parámetro	Significado
Frecuencia (f)	Cuántas veces se repite la señal por segundo
Período (T)	Duración de un ciclo completo
Tiempo en alto (T_{ON})	Tiempo durante el cual la señal está en nivel alto
Duty cycle (D)	Porcentaje del período en que la señal está en alto

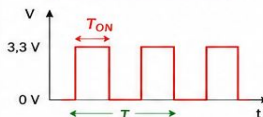
FORMAS DE ONDA PWM (misma frecuencia, diferente duty cycle)

Duty cycle = 25%



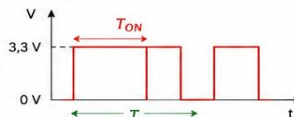
$$D = \frac{T_{ON}}{T} \cdot 100 = 25\%$$

Duty cycle = 50%



$$D = \frac{T_{ON}}{T} \cdot 100 = 50\%$$

Duty cycle = 75%

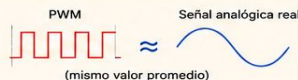


$$D = \frac{T_{ON}}{T} \cdot 100 = 75\%$$



IDEA CLAVE

PWM permite controlar el valor promedio entregado a una carga sin generar una tensión analógica real.



FÓRMULA

$$Duty = \frac{T_{ON}}{T} \cdot 100$$

Donde:

- T_{ON} : tiempo en alto
- T : período

EJEMPLOS DE USO



Control de brillo de un LED



Control de velocidad de un motor



Control de potencia entregada a una carga



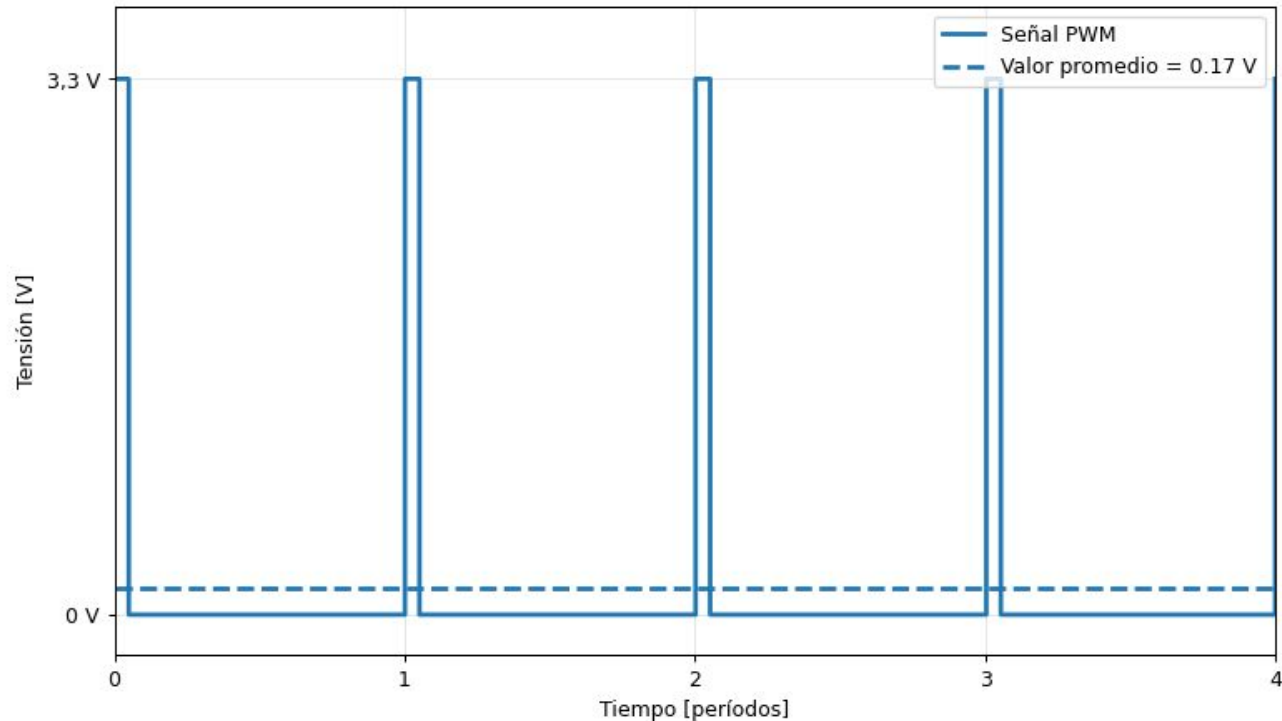
EN RESUMEN:

La PWM mantiene la frecuencia constante y varía el duty cycle para controlar la cantidad de energía promedio entregada a la carga.

0% duty → siempre en bajo
100% duty → siempre en alto

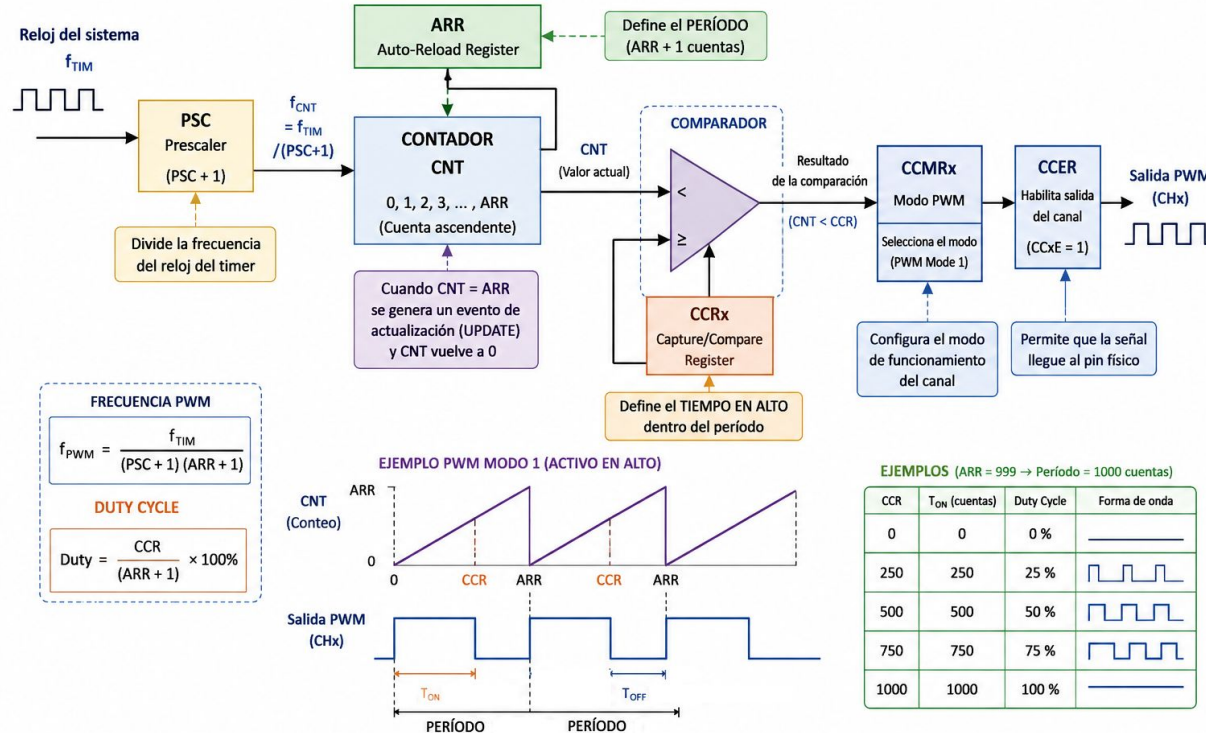
¿Qué es PWM?

PWM: al aumentar el duty cycle, aumenta el valor promedio
Duty cycle = 5 %



¿Cómo genera una PWM un temporizador?

GENERACIÓN DE PWM CON UN TIMER



Ejemplo - código fuente: ADC - PWM

Modos de uso de los canales del timer

Modo	Qué hace	Idea clave
Input Capture	Captura CNT ante un flanco externo	Medir tiempo
Output Compare	Compara CNT con CCR	Ejecutar acción en un instante
PWM	Genera señal periódica	ARR período, CCR duty
One-pulse	Genera un único pulso	Respuesta temporizada

Canales de Temporizadores

Timer	Tipo	Resolución	Canales
TIM1,TIM8	Avanzado	16 bits	4
TIM2,TIM5	General completo	32 bits	4
TIM3,TIM4	General completo	16 bits	4
TIM9,TIM12	General compacto	16 bits	2
TIM10,TIM11,TIM13,TIM14	General compacto	16 bits	1
TIM6,TIM7	Básico	16 bits	0